

Arquitectura de ordenadores 2024/2025

Práctica 2

Jorge Jiménez Oropesa, Antonio Moroño Moreno

Grupo 1322, pareja 15

Ejercicio 1

Apartado 1

Modelo de CPU:

```
$ cat /proc/cpuinfo | grep "model name" | head -1  
  
model name      : Intel(R) Core(TM) i5-8500 CPU @ 3.00GHz
```

Las instrucciones SIMD admitidas pueden ser identificadas con los siguientes flags

```
$ cat /proc/cpuinfo | grep "flags" | head -1  
  
flags           : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge  
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe  
syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts  
rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq  
dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm  
pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave  
avx fl6c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb  
invpcid_single pti ssbd ibrs ibpb stibp tpr_shadow flexpriority ept  
vpid ept_ad fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid mpx  
rdseed adx smap clflushopt intel_pt xsaveopt xsavec xgetbv1 xsaves  
dtherm ida arat pln pts hwp hwp_notify hwp_act_window hwp_epp vnmi  
md_clear flush_lld arch_capabilities
```

A continuación se muestra la versión del compilador.

```
$ cc --version  
cc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0  
Copyright (C) 2021 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There  
is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A  
PARTICULAR PURPOSE.
```

Para el siguiente punto de este apartado revisamos el código y comprobamos que el programa en C hace lo siguiente:

1. Define el tamaño del array 'ARRAY_SIZE' como 2048 y el número de pruebas 'NUMBER_OF_TRIALS' como 1.000.000.

2. Declara tres arrays estáticos de tipo 'double' ('a', 'b') y una variable 'c'.
3. En la función 'main', inicializa dos arrays 'a' y 'b' con valores secuenciales.
4. Ejecuta un bucle anidado:
 - El bucle externo se ejecuta 'NUMBER_OF_TRIALS' veces.
 - El bucle interno recorre los elementos del array 'a' y 'b', realizando una operación matemática y acumulando el resultado en 'c'.

Resumidamente, el código inicializa una serie de arrays, y luego realiza una operación matemática repetidamente, acumulando los resultados en la variable 'c'.

Para el siguiente punto vamos a compilar el programa con las siguientes opciones de compilación y vamos a comentar el informe obtenido.

```
$ gcc -O3 -march=native -fopt-info-vec-optimized -o simple2 simple2.c
simple2.c:26:21: optimized: loop vectorized using 32 byte vectors
simple2.c:19:17: optimized: loop vectorized using 32 byte vectors
```

Inspeccionando el informe de gcc, podemos ver que la vectorización del bucle se ha realizado correctamente ya que se muestra que se vectorizan dos bucles: el bucle que inicializa los valores de los datos y el bucle interno que realiza el cálculo principal:

La versión del gcc utilizada es la siguiente:

```
$ cc --version
cc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There
is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A
PARTICULAR PURPOSE.
```

Para el siguiente punto se compilará el programa con la vectorización deshabilitada, no se generará ningún informe porque no se ha vectorizado nada al tener la vectorización deshabilitada.

```
$ gcc -O3 -march=native -fno-tree-vectorize -fopt-info-vec-optimized
simple2.c -o simple2_no_vec
```

Ahora, vamos a comentar las diferencias entre el código ensamblador sin vectorizar y el vectorizado.

Opciones de compilación

`simple2_o3.s` ha sido generado mediante el comando

```
gcc -S -O3 -fno-tree-vectorize simple2.c
```

De modo que no utiliza vectorización.

`simple2_o3_native.s` se ha generado utilizando

```
gcc -O3 -march=native -fopt-info-vec-optimized -S -o simple2.s simple2.c
```

El código ensamblador generado, por tanto, está optimizado para la arquitectura específica de nuestro procesador, y emplea vectorización de bucles nativa.

Comparación de código ensamblador

Para la comparación, emplearemos la instrucción `diff simple2_o3.s simple2_o3_native.s -y -color`:

```

.file "simple2.c"
.text
.section .text.startup,"ax",@progbits
.p2align 4
.globl main
.type main, @function
main:
.LFB22:
.cfi_startproc
leaq b(%rip), %rdx
leaq a(%rip), %rcx
movl $4, %r8d
movl $1, %r9d
movd %r8d, %xmm4
movd %r9d, %xmm3
movq %rdx, %rax
movq %rcx, %rsi
movdqa .LC0(%rip), %xmm2
pshufd $0, %xmm4, %xmm4
pshufd $0, %xmm3, %xmm3
leaq 16384(%rdx), %rdi
.L2:
movdqa %xmm2, %xmm0
addq $32, %rax
padd %xmm4, %xmm2
addq $32, %rsi
cvtddq2pd %xmm0, %xmm1
movaps %xmm1, -32(%rax)
pshufd $238, %xmm0, %xmm1
padd %xmm3, %xmm0
cvtddq2pd %xmm1, %xmm1
movaps %xmm1, -16(%rax)
cvtddq2pd %xmm0, %xmm1
pshufd $238, %xmm0, %xmm0
cvtddq2pd %xmm0, %xmm0
movaps %xmm1, -32(%rsi)
movaps %xmm0, -16(%rsi)
cmpq %rdi, %rax
jne .L2
movsd c(%rip), %xmm3
movsd c(%rip), %xmm1
movl $1000000, %esi
unpcklpd %xmm3, %xmm3
.L3:
xorl %eax, %eax
.p2align 6
.p2align 4
.p2align 3
.L4:
movapd (%rcx,%rax), %xmm0
mulpd %xmm3, %xmm0
addpd (%rdx,%rax), %xmm0
addq $16, %rax
addsd %xmm0, %xmm1
unpckhpd %xmm0, %xmm0
addsd %xmm0, %xmm1
cmpq $16384, %rax
jne .L4
subl $1, %esi
jne .L3
movsd %xmm1, c(%rip)
xorl %eax, %eax
ret
.cfi_endproc
.LFE22:
.size main, .-main
.local c
.comm c,8,8
.local b
.comm b,16384,32
.local a
.comm a,16384,32
.section .rodata.cst16,"aM",@progbits,16
.align 16
.LC0:
.long 0
.long 1
.long 2
.long 3
.section .rodata.cst8,"aM",@progbits,8
.align 8
.LC4:
.long -611603343
.long 1072693352
.ident "GCC: (Debian 14.2.0-6) 14.2.0"
.section .note.GNU-stack,"",@progbits

.file "simple2.c"
.text
.section .text.startup,"ax",@progbits
.p2align 4
.globl main
.type main, @function
main:
.LFB22:
.cfi_startproc
xorl %eax, %eax
leaq b(%rip), %rcx
leaq a(%rip), %rdx
.L2:
pxor %xmm0, %xmm0
leal 1(%rax), %esi
cvtis2sdl %eax, %xmm0
movsd %xmm0, (%rcx,%rax,8)
pxor %xmm0, %xmm0
cvtis2sdl %esi, %xmm0
movsd %xmm0, (%rdx,%rax,8)
addq $1, %rax
cmpq $2048, %rax
jne .L2
movsd c(%rip), %xmm1
movsd .LC0(%rip), %xmm2
movl $1000000, %esi
.L3:
xorl %eax, %eax
.p2align 5
.p2align 4
.p2align 3
.L4:
movsd (%rdx,%rax), %xmm0
mulsd %xmm2, %xmm0
addsd (%rcx,%rax), %xmm0
addq $8, %rax
addsd %xmm0, %xmm1
cmpq $16384, %rax
jne .L4
subl $1, %esi
jne .L3
movsd %xmm1, c(%rip)
xorl %eax, %eax
ret
.cfi_endproc
.LFE22:
.size main, .-main
.local c
.comm c,8,8
.local b
.comm b,16384,32
.local a
.comm a,16384,32
.section .rodata.cst8,"aM",@progbits,8
.align 8
.LC0:
.long -611603343
.long 1072693352
.ident "GCC: (Debian 14.2.0-6) 14.2.0"
.section .note.GNU-stack,"",@progbits

```

1. Instrucciones vectoriales y no vectoriales:

- `simple2_o3.s` no utiliza instrucciones vectoriales debido a la opción `-fno-tree-vectorize`. Utiliza las versiones escalares, como `movsd`, `addsd` y `mulsd`
- `simple2_o3_native.s` utiliza instrucciones optimizadas y vectorizadas como `movapd`, `addpd` y `mulpd` debido a las opciones `-march=native` y `-fopt-info-vec-optimized`. Además,

2. Alineación y secciones de datos: Ambos códigos hacen uso de `.align`, pero lo hacen con valores distintos. Además, antes de LC0 podemos observar que el código optimizado añade un `.align` extra.

3. Complejidad: el código optimizado no sólo sustituye instrucciones escalares por sus versiones vectoriales, si no que además añade otras tantas instrucciones adicionales, como `unpcklpsd`, que hacen que el código resultante sea algo más complicado.

Estas diferencias reflejan cómo las opciones de compilación afectan las optimizaciones y el uso de instrucciones específicas del procesador.

Apartado 2

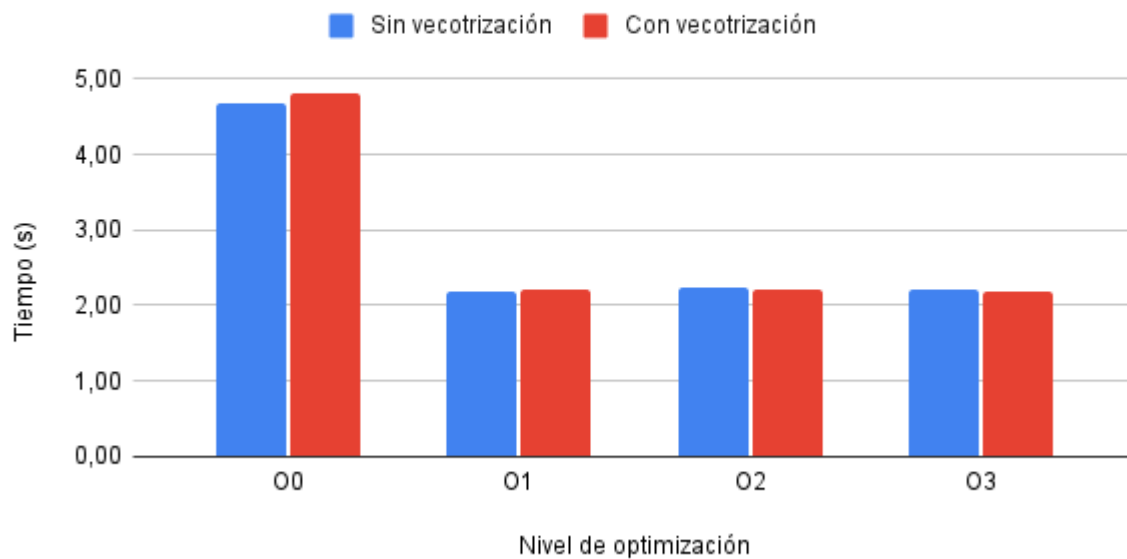
Para este apartado compilamos el programa `simple2.c` con distintas opciones de compilación y ejecutamos los binarios varias veces con un script de bash.

Número de ejecuciones por programa: `NUMBER_OF_TRIALS = 106`

Sin vectorización	Con vectorización
Opciones de compilación: <code>gcc simple2.c -o simple2base</code> Tiempo medio de ejecución: 4,6892349s	Opciones de compilación: <code>gcc -ftree-vectorize simple2.c -o simple2</code> Tiempo medio de ejecución: 4,8035529s
Opciones de compilación: <code>gcc -O1 -fno-tree-vectorize simple2.c -o simple2_O1_no_vect</code> Tiempo medio de ejecución: 2,1806939s	Opciones de compilación: <code>gcc -O1 -ftree-vectorize simple2.c -o simple2</code> Tiempo medio de ejecución: 2,2145817s
Opciones de compilación: <code>gcc -O2 -fno-tree-vectorize simple2.c -o simple2_O2_no_vect</code> Tiempo medio de ejecución: 2,2280631s	Opciones de compilación: <code>gcc -O2 -ftree-vectorize simple2.c -o simple2</code> Tiempo medio de ejecución: 2,2044695s
Opciones de compilación: <code>gcc -O3 -fno-tree-vectorize simple2.c -o simple2_O3_no_vect</code> Tiempo medio de ejecución: 2,2163614s	Opciones de compilación: <code>gcc -O3 -ftree-vectorize simple2.c -o simple2</code> Tiempo medio de ejecución: 2,1830953s

Comparación entre distintos niveles de optimización

Número de operaciones: 1.000.000



Para realizar estas mediciones ejecutamos 10 veces el programa y tomamos el promedio del tiempo de cada ejecución.

Sorprendentemente apenas hay diferencias entre el código con vectorización y sin vectorizar, es más, en ocasiones el código vectorizado parece ser más lento. Planteamos que puede que el número de operaciones sea demasiado bajo como para que la vectorización compense las instrucciones adicionales que se emplean para vectorizar el código.

Apartado 3

Ejecutando el comando: `gcc -c -Q -march=native --help=target`
nos muestra las opciones posibles para el flag march son las siguientes:

```
nocona core2 nehalem corei7 westmere sandybridge corei7-avx
ivybridge core-avx-i haswell core-avx2 broadwell skylake
skylake-avx512 cannonlake icelake-client rocketlake
icelake-server cascadelake tigerlake cooperlake sapphirerapids
emeraldrapids alderlake raptorlake meteorlake graniterapids
graniterapids-d arrowlake arrowlake-s lunarlake pantherlake
bonnell atom silvermont slm goldmont goldmont-plus tremont
gracemont sierraforest grandridge clearwaterforest knl knm
x86-64 x86-64-v2 x86-64-v3 x86-64-v4 eden-x2 nano nano-1000
nano-2000 nano-3000 nano-x2 eden-x4 nano-x4 lujiangzui yongfeng k8
k8-sse3 opteron opteron-sse3 athlon64 athlon64-sse3 athlon-fx
amdfam10 barcelona bdver1 bdver2 bdver3 bdver4 znver1 znver2 znver3
znver4 znver5 btver1 btver2 native
```

Para el siguiente punto vamos a realizar tres crosscompilaciones: una para una arquitectura SSE (**k8-sse3**), una para una arquitectura AVX (**corei7-avx**) y otra para una arquitectura AVX512 (**skylake-avx512**). Se recomienda mirar los archivos en la carpeta EJ1/asm para entender mejor la comparación:

- **k8-sse3 vs corei7-avx**

28,29c28,29

```
<    movlpd .LC0(%rip), %xmm0
<    movsd  %xmm0, -16(%rbp)
```

```
>    vmovsd .LC0(%rip), %xmm0
>    vmovsd %xmm0, -16(%rbp)
```

45c45

```
<    cvtsi2sdl    -4(%rbp), %xmm0
```

```
>    vcvtsi2sdl    -4(%rbp), %xmm0, %xmm0
```

50c50

```
<    movsd  %xmm0, (%rdx,%rax)
```

```
>    vmovsd %xmm0, (%rdx,%rax)
```

52,53c52,53

```
<    incl    %eax
<    cvtsi2sdl    %eax, %xmm0
```

```
>    addl    $1, %eax
>    vcvtsi2sdl    %eax, %xmm0, %xmm0
```

58,59c58,59

```
<    movsd  %xmm0, (%rdx,%rax)
<    incl    -4(%rbp)
```

```
>    vmovsd %xmm0, (%rdx,%rax)
>    addl    $1, -4(%rbp)
```

73,75c73,74

```
<    movlpd (%rdx,%rax), %xmm0
<    movsd  %xmm0, %xmm1
<    mulsd  -16(%rbp), %xmm1
```

```
>    vmovsd (%rdx,%rax), %xmm0
>    vmulsd -16(%rbp), %xmm0, %xmm1
```

80,85c79,84

```
<    movlpd (%rdx,%rax), %xmm0
<    addsd  %xmm0, %xmm1
<    movlpd c(%rip), %xmm0
<    addsd  %xmm1, %xmm0
<    movsd  %xmm0, c(%rip)
<    incl    -4(%rbp)
```

```
>    vmovsd (%rdx,%rax), %xmm0
>    vaddsd %xmm0, %xmm1, %xmm1
>    vmovsd c(%rip), %xmm0
>    vaddsd %xmm0, %xmm1, %xmm0
>    vmovsd %xmm0, c(%rip)
>    addl    $1, -4(%rbp)
```

89c88

```
<    incl    -8(%rbp)
```

```

>    addl    $1, -8(%rbp)
108c107
<    cvtsi2sdq    %rdx, %xmm1
---
>    vcvtsi2sdq    %rdx, %xmm1, %xmm1
112,117c111,117
<    cvtsi2sdq    %rdx, %xmm0
<    movlpd    .LC2(%rip), %xmm2
<    divsd    %xmm2, %xmm0
<    addsd    %xmm1, %xmm0
<    movsd    %xmm0, -24(%rbp)
<    movlpd    -24(%rbp), %xmm0
---
>    vcvtsi2sdq    %rdx, %xmm0, %xmm0
>    vmovsd    .LC2(%rip), %xmm2
>    vdivsd    %xmm2, %xmm0, %xmm0
>    vaddsd    %xmm0, %xmm1, %xmm0
>    vmovsd    %xmm0, -24(%rbp)
>    movq    -24(%rbp), %rax
>    vmovq    %rax, %xmm0

```

Diferencias:

Prefijo de Instrucciones:

- SSE: Utiliza instrucciones como movsd, cvtsi2sdl, mulsd, addsd.
- AVX: Utiliza instrucciones con prefijo v, como vmovsd, vcvtsi2sdl, vmulsd, vaddsd.
- Registros:
 - SSE: Opera principalmente con registros xmm.
 - AVX: También opera con registros xmm, pero permite el uso de registros extendidos ymm para operaciones más eficientes.
- Operaciones:
 - SSE: Las instrucciones SSE realizan operaciones en un solo registro de destino.
 - AVX: Las instrucciones AVX pueden especificar tanto el registro de origen como el de destino, permitiendo operaciones más flexibles y eficientes.

● k8-sse vs skylake-avx512

```

28,29c28,29
<    movlpd    .LC0(%rip), %xmm0
<    movsd    %xmm0, -16(%rbp)
---
>    vmovsd    .LC0(%rip), %xmm0
>    vmovsd    %xmm0, -16(%rbp)
45c45
<    cvtsi2sdl    -4(%rbp), %xmm0
---
>    vcvtsi2sdl    -4(%rbp), %xmm0, %xmm0
50c50
<    movsd    %xmm0, (%rdx,%rax)
---
>    vmovsd    %xmm0, (%rdx,%rax)
53c53
<    cvtsi2sdl    %eax, %xmm0
---
>    vcvtsi2sdl    %eax, %xmm0, %xmm0
58c58

```

```

<    movsd  %xmm0, (%rdx,%rax)
---
>    vmovsd %xmm0, (%rdx,%rax)
73,75c73,74
<    movlpd (%rdx,%rax), %xmm0
<    movsd  %xmm0, %xmm1
<    mulsd  -16(%rbp), %xmm1
---
>    vmovsd (%rdx,%rax), %xmm0
>    vmulsd -16(%rbp), %xmm0, %xmm0
80,84c79,83
<    movlpd (%rdx,%rax), %xmm0
<    addsd  %xmm0, %xmm1
<    movlpd c(%rip), %xmm0
<    addsd  %xmm1, %xmm0
<    movsd  %xmm0, c(%rip)
---
>    vmovsd (%rdx,%rax), %xmm1
>    vaddsd %xmm1, %xmm0, %xmm0
>    vmovsd c(%rip), %xmm1
>    vaddsd %xmm1, %xmm0, %xmm0
>    vmovsd %xmm0, c(%rip)
108c107
<    cvtsi2sdq    %rdx, %xmm1
---
>    vcvtsi2sdq    %rdx, %xmm1, %xmm1
112,117c111,117
<    cvtsi2sdq    %rdx, %xmm0
<    movlpd  .LC2(%rip), %xmm2
<    divsd  %xmm2, %xmm0
<    addsd  %xmm1, %xmm0
<    movsd  %xmm0, -24(%rbp)
<    movlpd -24(%rbp), %xmm0
---
>    vcvtsi2sdq    %rdx, %xmm0, %xmm0
>    vmovsd  .LC2(%rip), %xmm2
>    vdivsd %xmm2, %xmm0, %xmm0
>    vaddsd %xmm0, %xmm1, %xmm0
>    vmovsd %xmm0, -24(%rbp)
>    movq   -24(%rbp), %rax
>    vmovq  %rax, %xmm0

```

Diferencias:

- Prefijo de Instrucciones:
 - SSE: Utiliza instrucciones como movsd, cvtsi2sdl, mulsd, addsd.
 - AVX-512: Utiliza instrucciones con prefijo v, como vmovsd, vcvtsi2sdl, vmulsd, vaddsd.
- Registros:
 - SSE: Opera principalmente con registros xmm (128 bits).
 - AVX-512: Opera con registros zmm (512 bits), permitiendo operaciones en vectores más grandes y más eficientes.
- Operaciones:
 - SSE: Las instrucciones SSE realizan operaciones en un solo registro de destino.
 - AVX-512: Las instrucciones AVX-512 pueden especificar tanto el registro de origen como el de destino, permitiendo operaciones más flexibles y eficientes.

- **corei7-avx vs skylake-avx512**

```

52c52
<    addl    $1, %eax
---
>    incl    %eax
59c59
<    addl    $1, -4(%rbp)
---
>    incl    -4(%rbp)
74c74
<    vmulsd  -16(%rbp), %xmm0, %xmm1
---
>    vmulsd  -16(%rbp), %xmm0, %xmm0
79,82c79,82
<    vmovsd  (%rdx,%rax), %xmm0
<    vaddsd  %xmm0, %xmm1, %xmm1
<    vmovsd  c(%rip), %xmm0
<    vaddsd  %xmm0, %xmm1, %xmm0
---
>    vmovsd  (%rdx,%rax), %xmm1
>    vaddsd  %xmm1, %xmm0, %xmm0
>    vmovsd  c(%rip), %xmm1
>    vaddsd  %xmm1, %xmm0, %xmm0
84c84
<    addl    $1, -4(%rbp)
---
>    incl    -4(%rbp)
88c88
<    addl    $1, -8(%rbp)
---
>    incl    -8(%rbp)

```

Diferencias

- Prefijo de Instrucciones:
 - AVX: Utiliza instrucciones con prefijo v, como vmovsd, vcvtsi2sdl, vmulsd, vaddsd.
 - AVX-512: También utiliza instrucciones con prefijo v, pero puede operar con registros más grandes (zmm).
- Registros:
 - AVX: Opera con registros xmm (128 bits) y ymm (256 bits).
 - AVX-512: Opera con registros zmm (512 bits), permitiendo operaciones en vectores más grandes y más eficientes.
- Operaciones:
 - AVX: Las instrucciones AVX realizan operaciones en registros de 128 o 256 bits.
 - AVX-512: Las instrucciones AVX-512 pueden realizar operaciones en registros de 512 bits, permitiendo un mayor paralelismo y eficiencia.

Ejercicio 2

Creemos el archivo `simple2_intrinsics.c` tal y como se describe en el enunciado y procedemos a compilarlo y ejecutarlo para ver el resultado es el mismo que en `simple2`

Opciones de compilación:

```
gcc -mavx -mfma simple2_intrinsics.c -o simple2_intrinsics
```

Nótese que las opciones `-mavx` y `-mfma` son necesarias:

- `-mavx`: Permite al compilador generar instrucciones AVX para las operaciones con los tipos de datos y funciones intrínsecas
- `-mfma`: habilitar el uso de las instrucciones FMA que permiten realizar una operación de multiplicación seguida de una suma en una sola instrucción (como hacemos en nuestro código)

Estas opciones de compilación están implícitas en `-march=native` (siempre que nuestro procesador permita instrucciones AVX y FMA)

Apartado 1

Código para la vectorización del bucle de inicialización de los arrays usando intrínsecos:

```
/* Populate A and B arrays */
__m256d vb = _mm256_set_pd(0,1,2,3);
__m256d va = _mm256_set_pd(1,2,3,4);

for (i=0; i < ARRAY_SIZE; i+=4){
    __m256d sumando = _mm256_set1_pd(i);
    _mm256_store_pd((double*)&a[i], _mm256_add_pd(sumando,va));
    _mm256_store_pd((double*)&b[i], _mm256_add_pd(sumando,vb));
}
```

El código comienza inicializando dos vectores de 256 bits `va` y `vb` que contienen 4 doubles de 8 bytes. A continuación se establecen los parámetros del `for` con un iterador `i` que va desde 0 hasta `ARRAY_SIZE=2048` de 4 en 4.

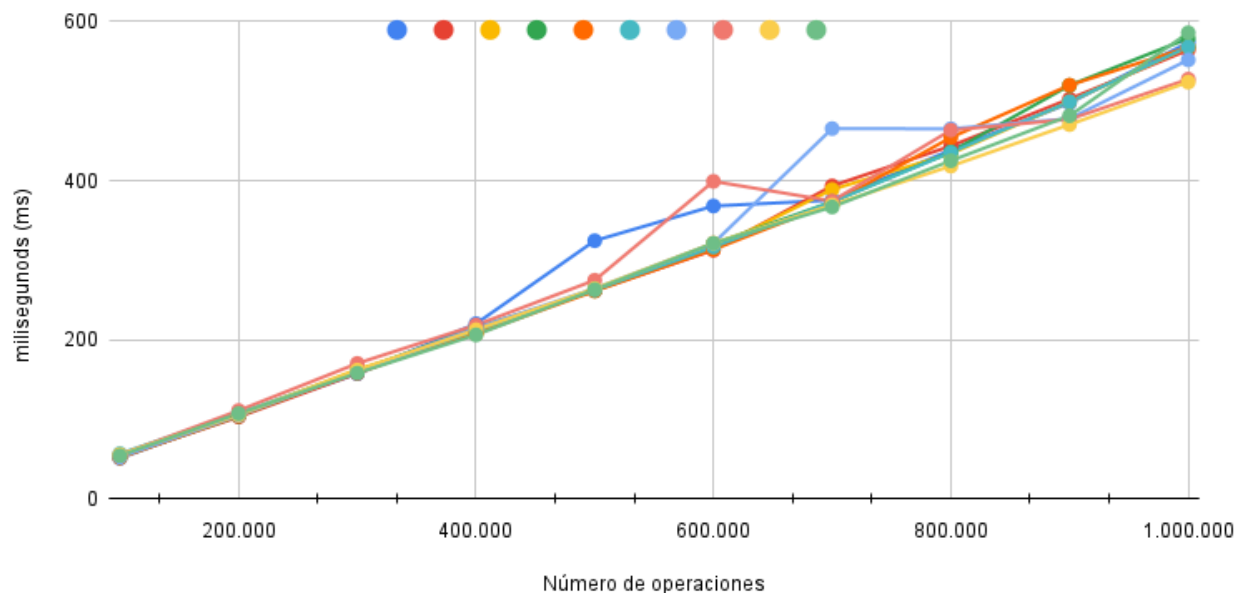
El iterador avanza de 4 en 4 ya que en cada iteración del bucle vamos a inicializar un vector de 256 bits con 4 doubles que van a tener el valor de `i`, después vamos a sumar los vectores `va` y `vb` con el vector `sumando` y vamos a guardar el resultado de las suma (dos vector de 4 números) en las posiciones adyacentes a `va` y `vb` respectivamente. Como guardamos 4 doubles en `a` y en `b` avanzamos de 4 en 4.

Apartado 2

Primero realizamos 10 mediciones sobre el código `simple2_intrinsics` incrementando el número de pruebas de 100.000 desde 100.000 hasta 1.000.000 para ilustrar la dispersión de los tiempos de ejecución.

Tiempo de ejecucion de `simple2_intrinsics`

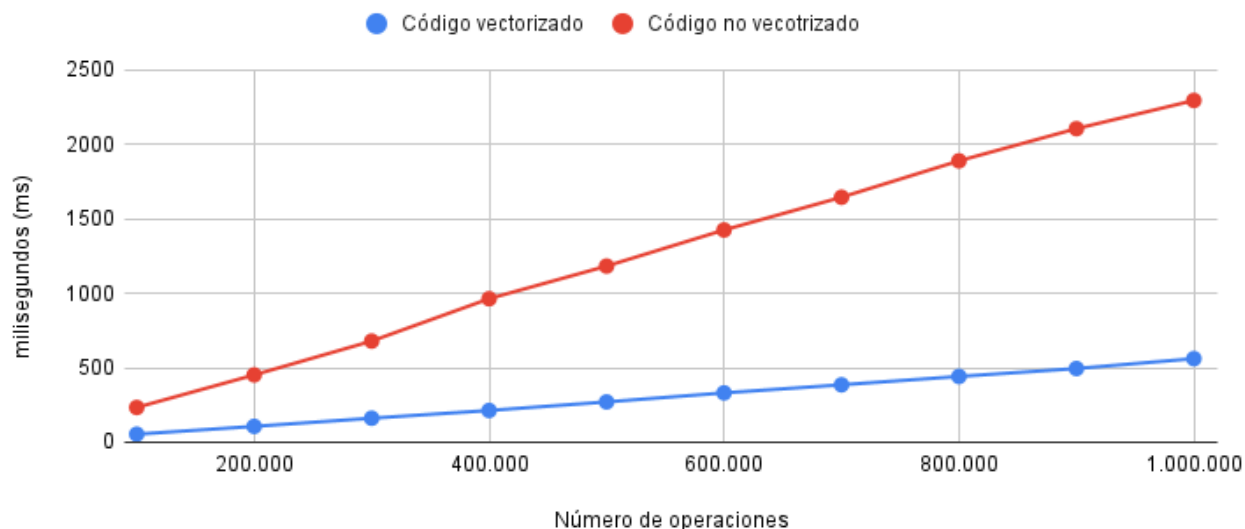
Cada color es una prueba de ejecucion



Como se puede ver obtenemos variaciones de hasta 100 ms en los picos más extremos aunque se ve una clara correlación entre todas las ejecuciones como es lógico.

Para la comparación entre el tiempo del código vectorizado y el código sin vectorizar se hizo la media de estas 10 mediciones para cada número de operaciones, y se hizo lo mismo para el código sin vectorizar. Estos fueron los resultados:

Comparación de tiempo entre `simple2` y `simple2_intrinsics`



Opciones de compilación:

- **Vectorizado:** `gcc -march=native -O3 simple2_intrinsics.c -o simple2_intrinsics`
- **Sin vectorizar:** `gcc -march=native -O3 -fno-tree-vectorize simple2.c -o simple2`

Número de operaciones	Vectorizado (ms)	No vectorizado (ms)	Rendimiento
100.000	53,80310	233,19320	4,334196357
200.000	106,21240	451,76480	4,253409206
300.000	161,03700	679,06290	4,216812906
400.000	212,95160	963,52320	4,524611226
500.000	270,40700	1.182,51910	4,373108315
600.000	331,10200	1.425,00190	4,303815441
700.000	385,37850	1.644,86600	4,268183098
800.000	441,49240	1.889,31020	4,279371967
900.000	494,63890	2.105,55740	4,256756596
1.000.000	561,00230	2.294,82430	4,09057913

Rendimiento medio: 4,290084424

El rendimiento es próximo al valor teórico que es 4 porque estamos trabajando con vectores de 4 elementos, es decir, por cada instrucción estamos realizando 4 operaciones.

Ejercicio 3

Apartado 1

E1.1 Considere la función `_mm256_srlv_epi64`. Incluso si no sabe lo que significa `srlv`, ¿Qué se puede decir de la salida y de los argumentos de entrada de la función?

Toma como entrada un vector de 256 bits formado por 4 enteros con signo de 64 bits, y devuelve un vector de enteros de 256 bits

E1.2 Considere la función `_mm_testnzc_ps`. Incluso si no sabe lo que significa `testnzc`, ¿Qué se puede decir de la salida y de los argumentos de entrada de la función?

Toma como entrada un vector de 128 bits formado por 4 floats (de 32 bits) y devuelve otro vector de floats de 128 bits

E1.3 En un registro SIMD de 128 bit, ¿Cuántos enteros se pueden almacenar?

Depende del tamaño del entero. Se pueden almacenar:

16 enteros de 8 bits (byte)

8 enteros de 16 bits (short)

4 enteros de 32 bits (32-bit integer)

2 enteros de 64 bits (32-bit integer)

E1.4 En un registro SIMD de 128 bit, ¿Cuántos números reales de precisión simple (float) se pueden almacenar?

Se pueden almacenar 4 floats, ya que cada uno ocupa 32 bits

E1.5 En un registro SIMD de 128 bit, ¿Cuántos números reales de doble precisión (double) se pueden almacenar?

Se pueden almacenar 2 doubles, ya que cada uno ocupa 64 bits

E1.6 ¿Cuántos números reales de doble precisión pueden almacenar?

Se pueden almacenar 32 números reales de doble precisión, ya que cada uno ocupa 64 bits

E1.7 ¿Cuántos caracteres se pueden almacenar?

Se pueden almacenar 256 caracteres, ya que cada uno ocupa 8 bits

E1.8 ¿Cuántos píxeles se pueden almacenar? (un píxel son 3 bytes para almacenar rojo-verde y azul)

Se pueden almacenar 85 píxeles, ya que cada uno ocupa 24 bytes, y no se puede almacenar una fracción de píxel

E1.9 ¿Cuántos enteros se pueden almacenar en un registro YMM?

Depende del tamaño del entero. Se pueden almacenar:

32 enteros de 8 bits (byte)

16 enteros de 16 bits (short)

8 enteros de 32 bits (32-bit integer)

4 enteros de 64 bits (32-bit integer)

E1.10 Suponga un vector de 32 doubles, (double vect[32]), ¿Cuántos registros se necesitan para procesarlo utilizando instrucciones SIMD?

Se necesitan 8 registros de 256 bits, ya que 32 doubles ocupan (32*64) 2048 bits

E2.1 ¿Coinciden siempre los resultados SIMD con el secuencial?

Si bien la diferencia es mínima, los resultados no siempre coinciden exactamente entre la versión SIMD y la secuencial. Ejemplo de ello es la ejecución para el tamaño de vector N = 512212, puesto que en la cuarta iteración:

El resultado SIMD es 87452942668.000061

El resultado secuencial 1 es 87452942667.999390, y el resultado secuencial 2 es 87452942667.999527

E2.2 ¿Coincide los denominados secuencial 1 a secuencial 2?

No, no siempre coinciden

E2.3 ¿Cómo se justifican estos resultados?

Los valores decimales, ya sean float o double, tienen una precisión limitada, pues ocupan un número limitado de bytes. Esto hace que al realizar operaciones como la suma o la multiplicación, se puedan tener que realizar forzosamente pequeñas aproximaciones que, al ir siendo

arrastradas, dan lugar a un pequeño error. Esto es responsable de las diferencias que observamos

E2.4 Si son diferentes ¿Cuál es el resultado correcto?

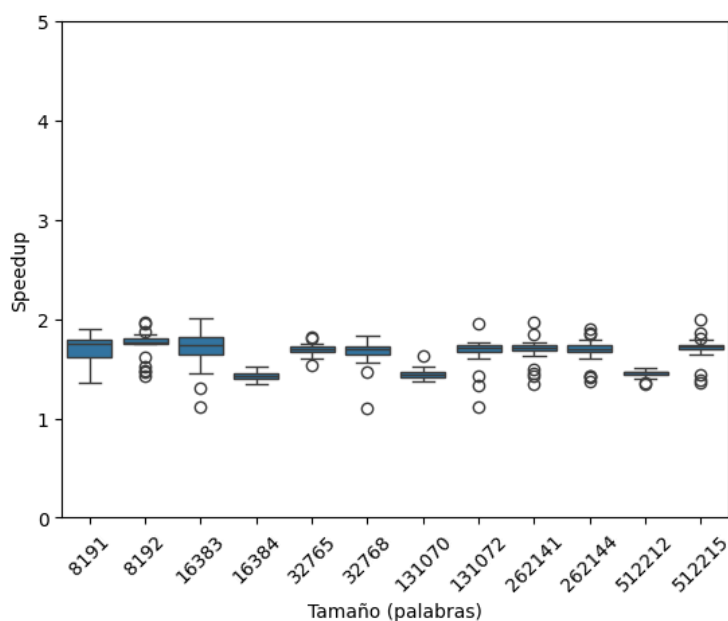
Todos los resultados son igual de "correctos", ya que su error respecto al resultado real es similar.

Apartado 2

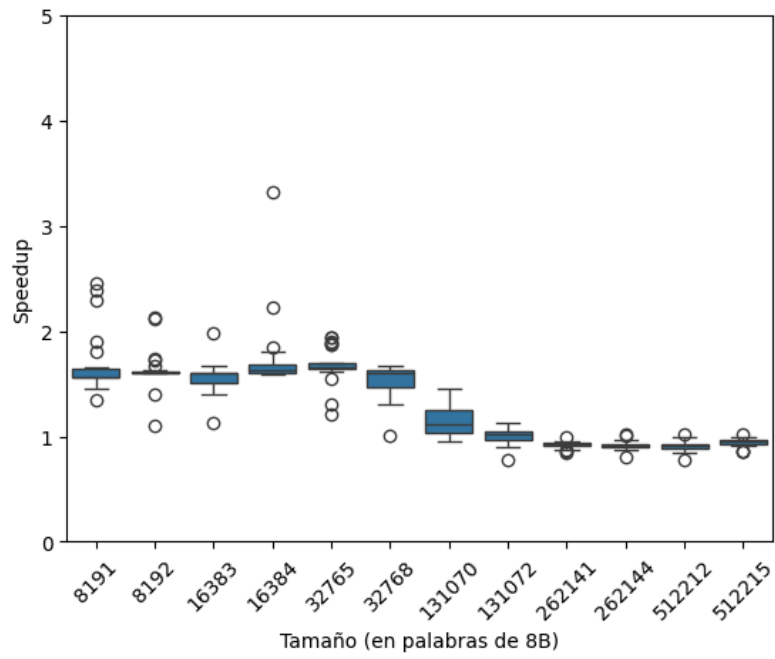
Para lograr un informe más completo y según se sugiere en el tutorial, hemos modificado los gráficos para que, en vez de limitarse a incluir información sobre el código compilado con las opciones de optimización O0 y O3, también se consideren O1 y O2.

Al reproducir el tutorial, estas han sido las estadísticas que hemos obtenido al ejecutar en Google Collab.

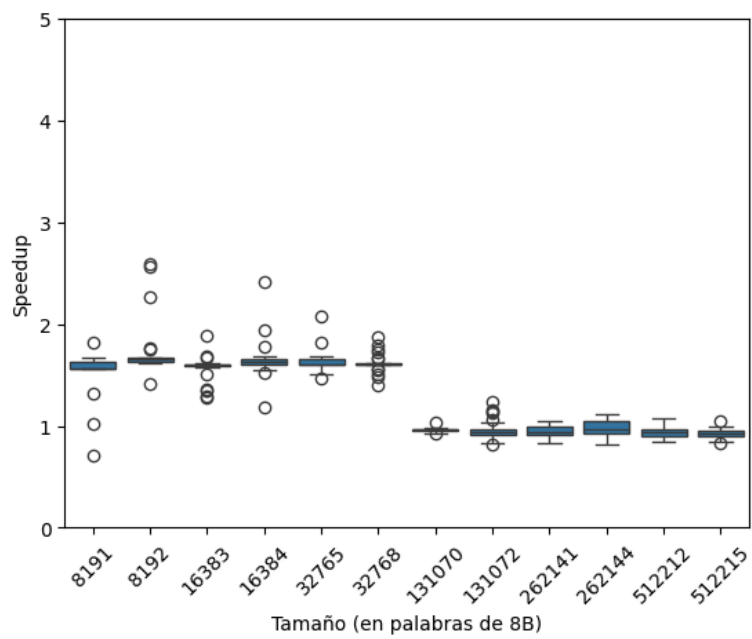
Sin optimización (O0):



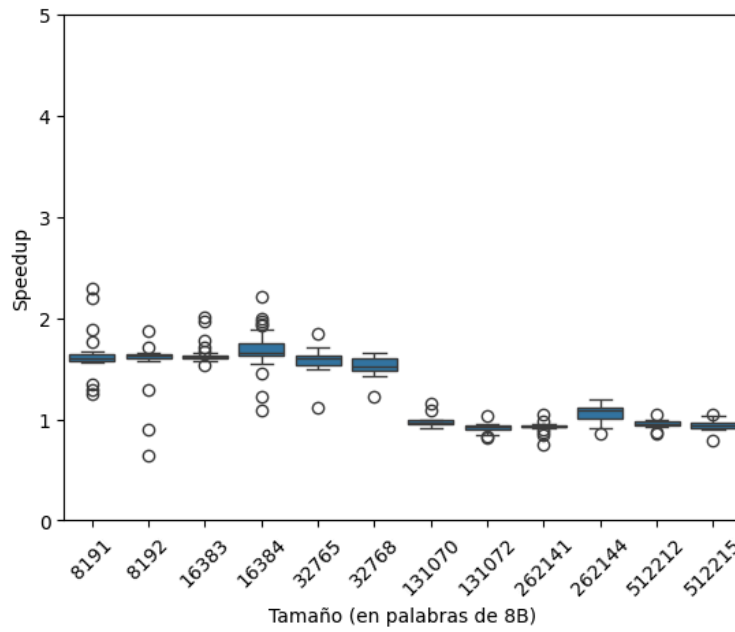
Con optimización O1:



Con optimización O2:

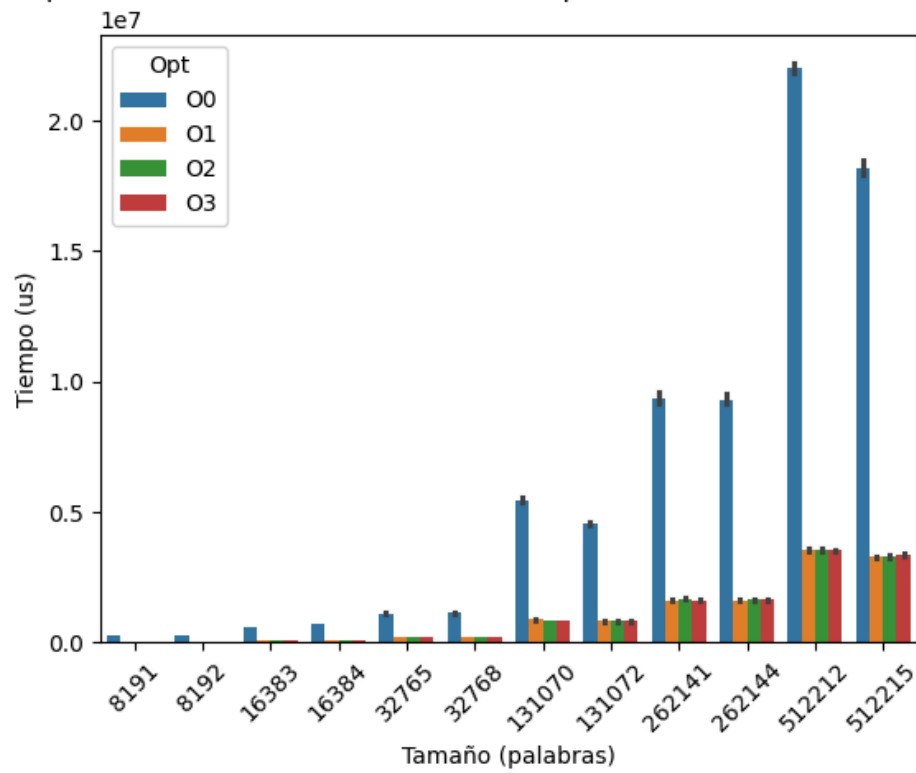


Con optimización O3:

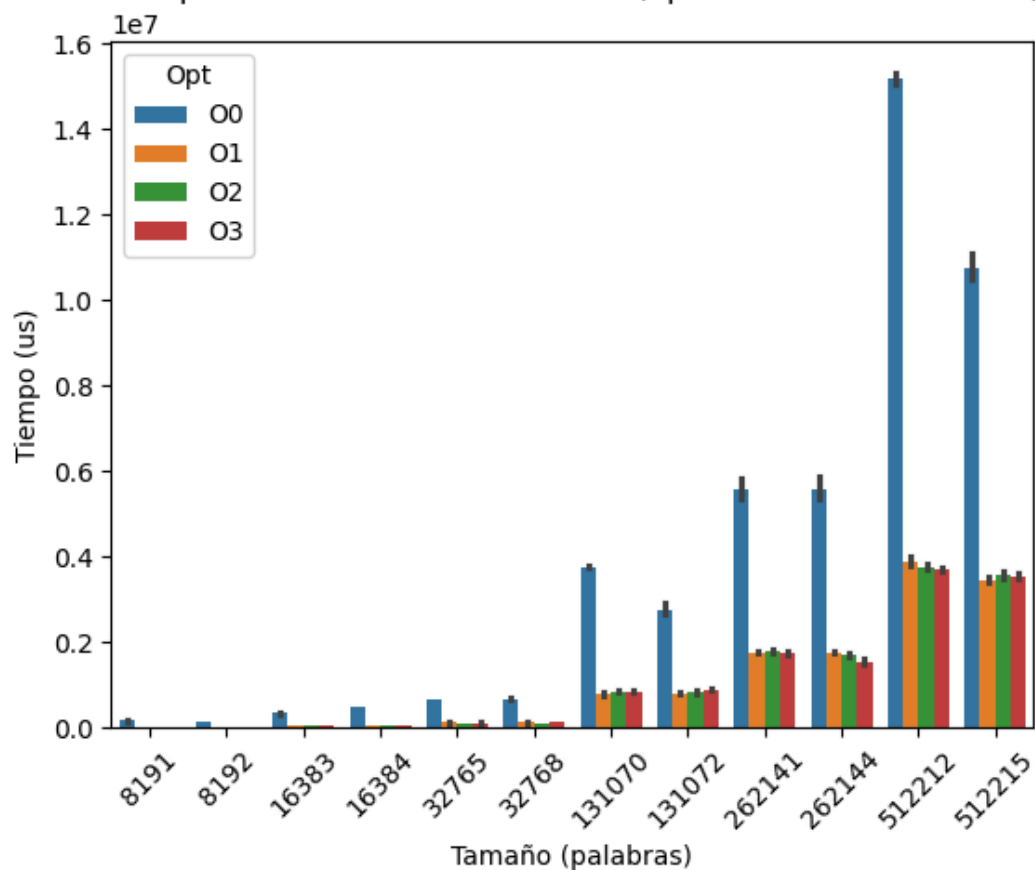


Y las tres siguientes gráficas comparan distintos aspectos de los cuatro modos de optimización:

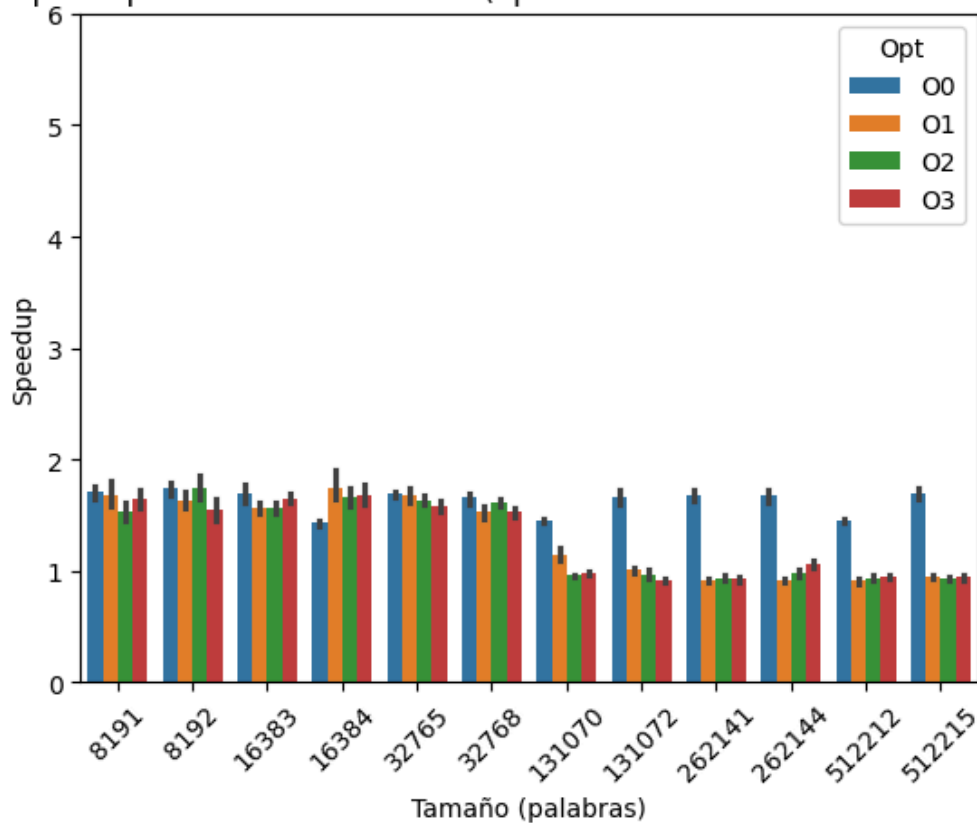
Tiempo (sin SIMD) vs tamaño del vector (optimización O0 vs O1 vs O2 vs O3)



Tiempo AVX vs tamaño del vector (optimización -O3 vs -O0)



Speedup vs tamaño del vector (optimización O0 vs O1 vs O2 vs O3)



En general, podemos comprobar que el mayor salto en rendimiento ocurre entre -O0 y -O1. Las diferencias entre -O1 y -O2 y -O3, sin embargo, no resultan significativas, y podemos observar que a veces unas resultan ligeramente más rápidas que otras para unos tamaños y para otros no. Estas pequeñas diferencias probablemente se deban a los pequeños errores a los que inevitablemente nos exponemos al realizar experimentos y mediciones en un entorno real.

Apartado 3

Hemos modificado el código proporcionado por el tutorial para que, en vez de calcular el producto escalar para vectores de 4 doubles, lo calcule para vectores de 8 floats, que también ocuparán 256 bits. A continuación se muestra el código:

```

#endif

// #define STATIC_MEM

#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <sys/time.h>
#include <immintrin.h>

typedef unsigned long long ull;

void initarray(float *a, ssize_t n) {
    for (ssize_t i = 0; i < n; i++) {
        a[i] = rand() / (rand() + 1);
    }
}

static inline float naive_dot_product(float *a, float *b, ssize_t n) {
    float res = 0.0;
    ssize_t i = 0;
    for (i = 0; i < n; i++) {
        res += a[i] * b[i];
    }
    return res;
}

static inline float reversed_dot_product(float *a, float *b, ssize_t n) {
    float res = 0.0;
    ssize_t i = 0;
    for (i = n - 1; i >= 0; i--) {
        res += a[i] * b[i];
    }
    return res;
}

/* Función para reducir los 8 floats de un vector de 256 bits a un solo valor float */
static inline float reduce_vector(__m256 input) {
    __m128 sum_high = _mm256_extractf128_ps(input, 1); // Extrae los 4 elementos superiores
    __m128 sum_low = _mm256_castps256_ps128(input);    // Extrae los 4 elementos inferiores
    __m128 sum = _mm_add_ps(sum_low, sum_high);        // Suma los dos bloques de 4 elementos

    // Realiza una suma horizontal en sum, siguiendo la filosofía de reduce_vector1
    __m128 temp = _mm_hadd_ps(sum, sum);
    return ((float*)&temp)[0] + ((float*)&temp)[2];
}

static inline float avx_dot_product(const float *a, const float *b, ssize_t n) {
    __m256 sum_vec = _mm256_setzero_ps(); // Cambiado a __m256

    /* Add up partial dot-products in blocks of 256 bits */
    for (ssize_t ii = 0; ii < n; ii += 8) {
        __m256 x = _mm256_load_ps(a + ii);
        __m256 y = _mm256_load_ps(b + ii);
        __m256 z = _mm256_mul_ps(x, y);
        sum_vec = _mm256_add_ps(sum_vec, z);
    }

    /* Find the partial dot-product for the remaining elements after
     * dealing with all 256-bit blocks. */
    float final = 0.0;
    for (ssize_t ii = n - (n % 8); ii < n; ++ii)
        final += a[ii] * b[ii];

    return reduce_vector(sum_vec) + final; // Llama a reduce_vector
}

int main(int argc, char **argv) {
    if (argc != 2) {
        printf(" uso: dot_product dotproductdata.txt \n");
        return 0;
    }

    const char *filename = argv[1];
    FILE *fp = fopen(filename, "a");
    if (fp == NULL) {
        perror(filename);
        return -1;
    }

#ifdef STATIC_MEM
    __attribute__((aligned(32))) float a[ARRAYSIZE], b[ARRAYSIZE];
#else

```

```

float *a, *b;
a = aligned_alloc(256, ARRAYSIZE * sizeof(float));
b = aligned_alloc(256, ARRAYSIZE * sizeof(float));
#endif

struct timespec ini, fin;
ull time_naive = 0, time_avx = 0;
initarray(a, ARRAYSIZE);
initarray(b, ARRAYSIZE);
float answers_naive[REPS] = {0}, answers_avx[REPS] = {0};
float answer_naive = 0, answer_avx = 0;
double speedup = 0;

for (ull iter = 0; iter < ITER; iter++) {
#define CLOCK_METHOD CLOCK_PROCESS_CPUTIME_ID

    clock_gettime(CLOCK_METHOD, &ini);
    for (ull r = 0; r < REPS; r++)
        answers_naive[r] = naive_dot_product(a, b, ARRAYSIZE);
    clock_gettime(CLOCK_METHOD, &fin);

    for (ull r = 0; r < REPS; r++)
        answer_naive += answers_naive[r];
    answer_naive = answer_naive / REPS;
    time_naive = (fin.tv_sec * 1000000000 + fin.tv_nsec) - (ini.tv_sec * 1000000000 +
    ini.tv_nsec);

    clock_gettime(CLOCK_METHOD, &ini);
    for (ull r = 0; r < REPS; r++)
        answers_avx[r] = avx_dot_product(a, b, ARRAYSIZE);
    clock_gettime(CLOCK_METHOD, &fin);

    for (ull r = 0; r < REPS; r++)
        answer_avx += answers_avx[r];
    answer_avx = answer_avx / REPS;
    time_avx = (fin.tv_sec * 1000000000 + fin.tv_nsec) - (ini.tv_sec * 1000000000 +
    ini.tv_nsec);

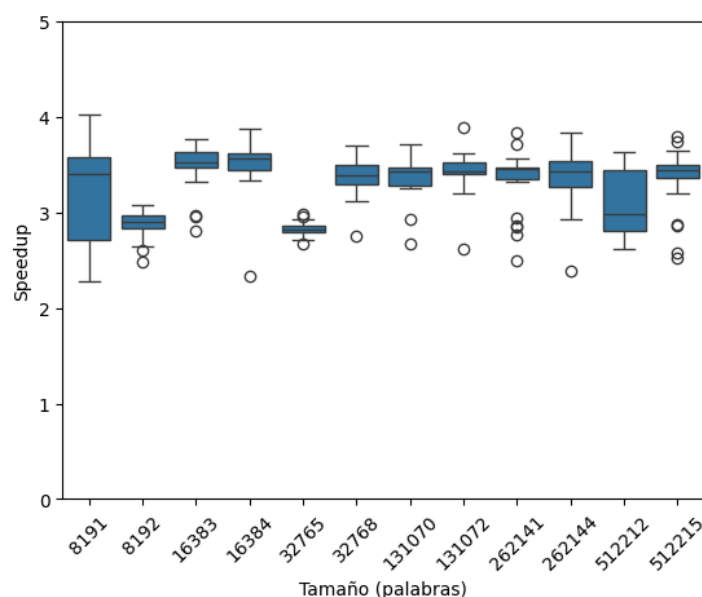
    speedup = ((double)time_naive) / ((double)time_avx);
    fprintf(fp, "%llu %u %lf %llu %lf %llu %lf\n", iter, ARRAYSIZE, answer_naive,
    time_naive, answer_avx, time_avx, speedup);
}

#ifdef STATIC_MEM
free(a);
free(b);
#endif
fclose(fp);
}

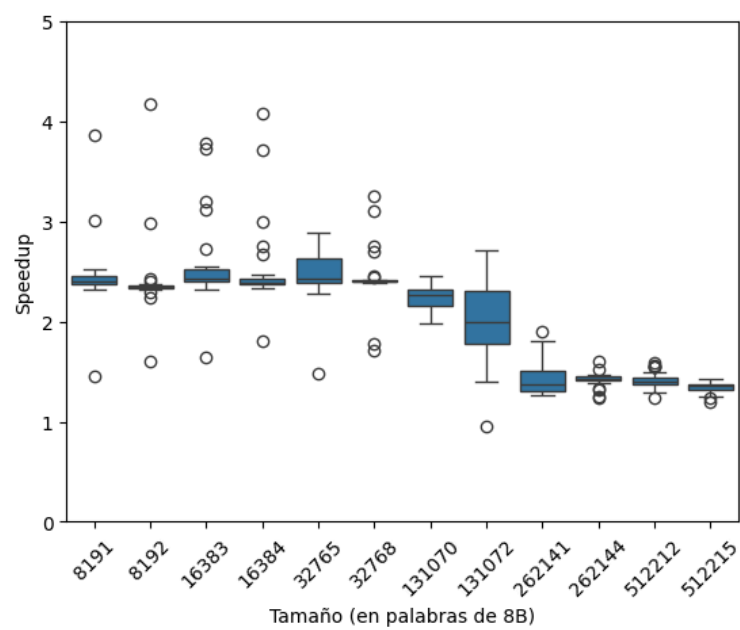
```

De nuevo, hemos elaborado estadísticas a partir de la ejecución del código:

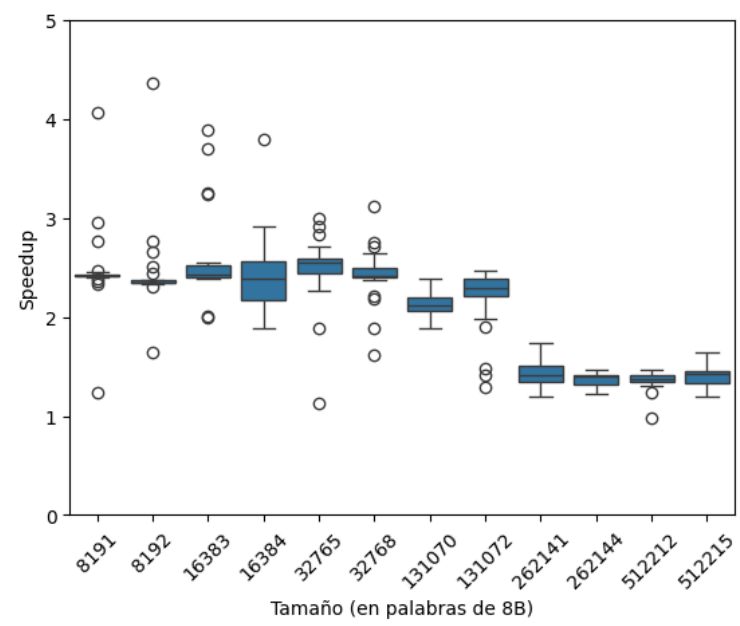
Sin optimización (O0):



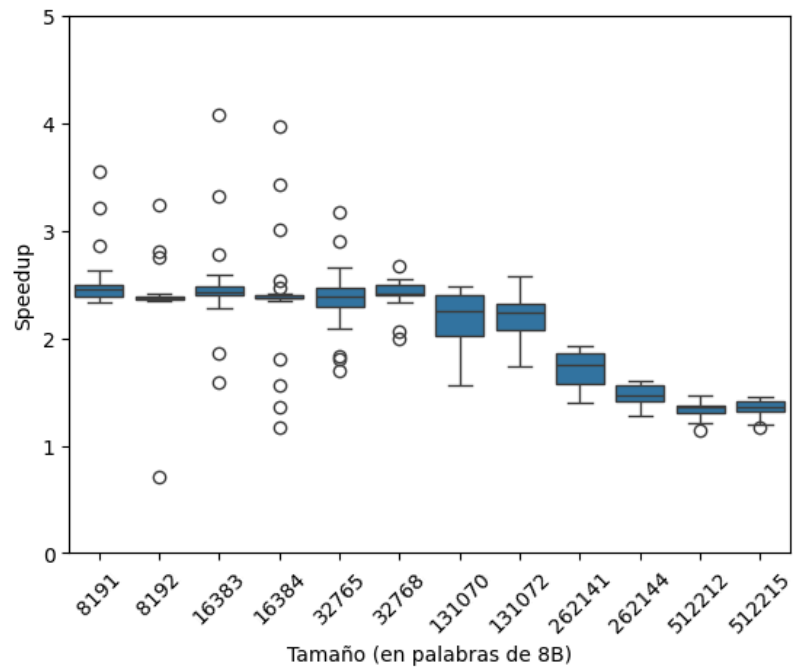
Con optimización (O1):



Con optimización (O2):

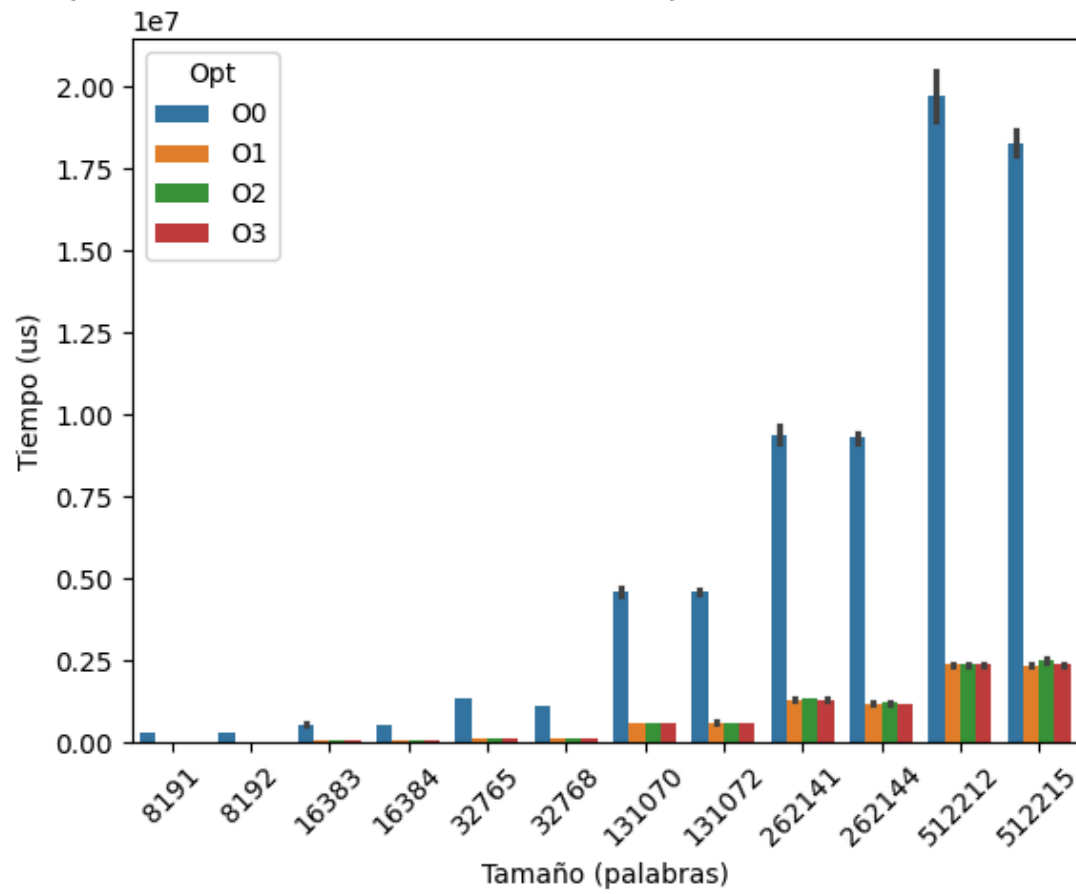


Con optimización (O3):

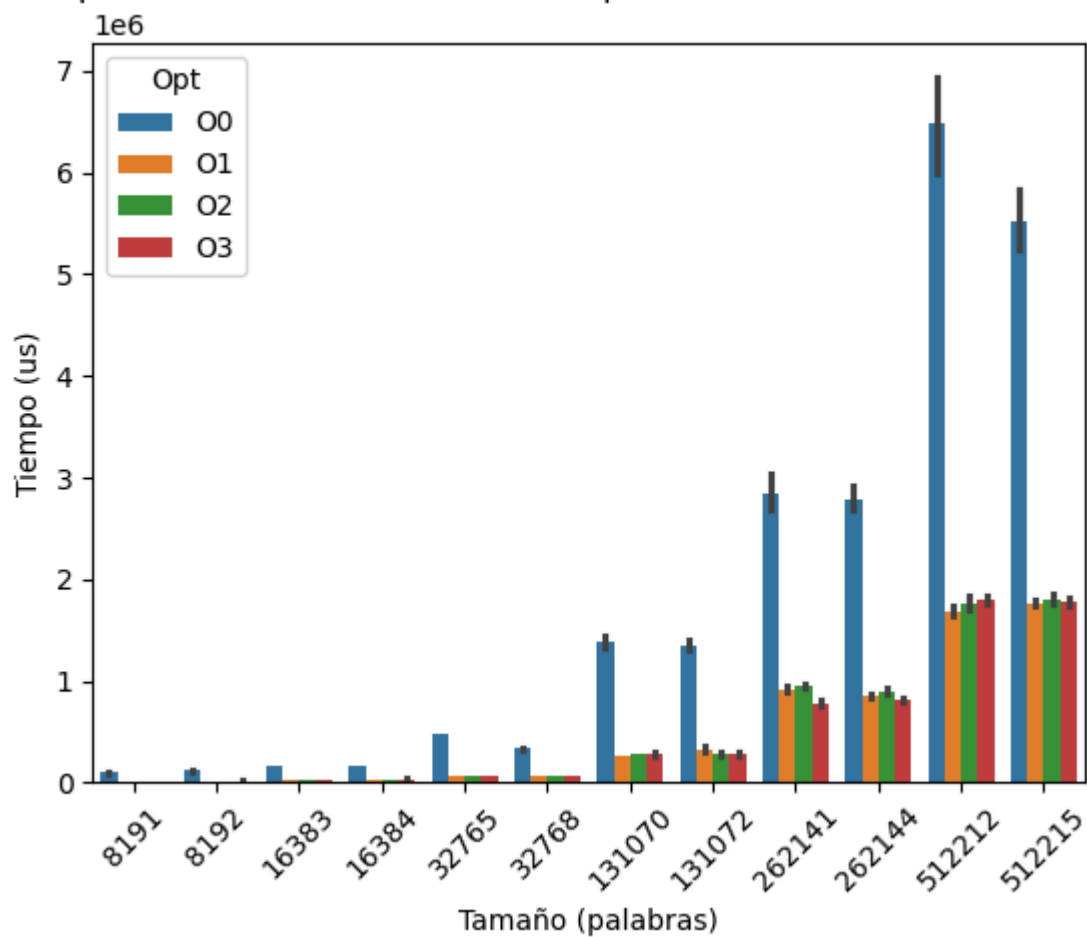


Y las tres siguientes gráficas comparan distintos aspectos de los cuatro modos de optimización:

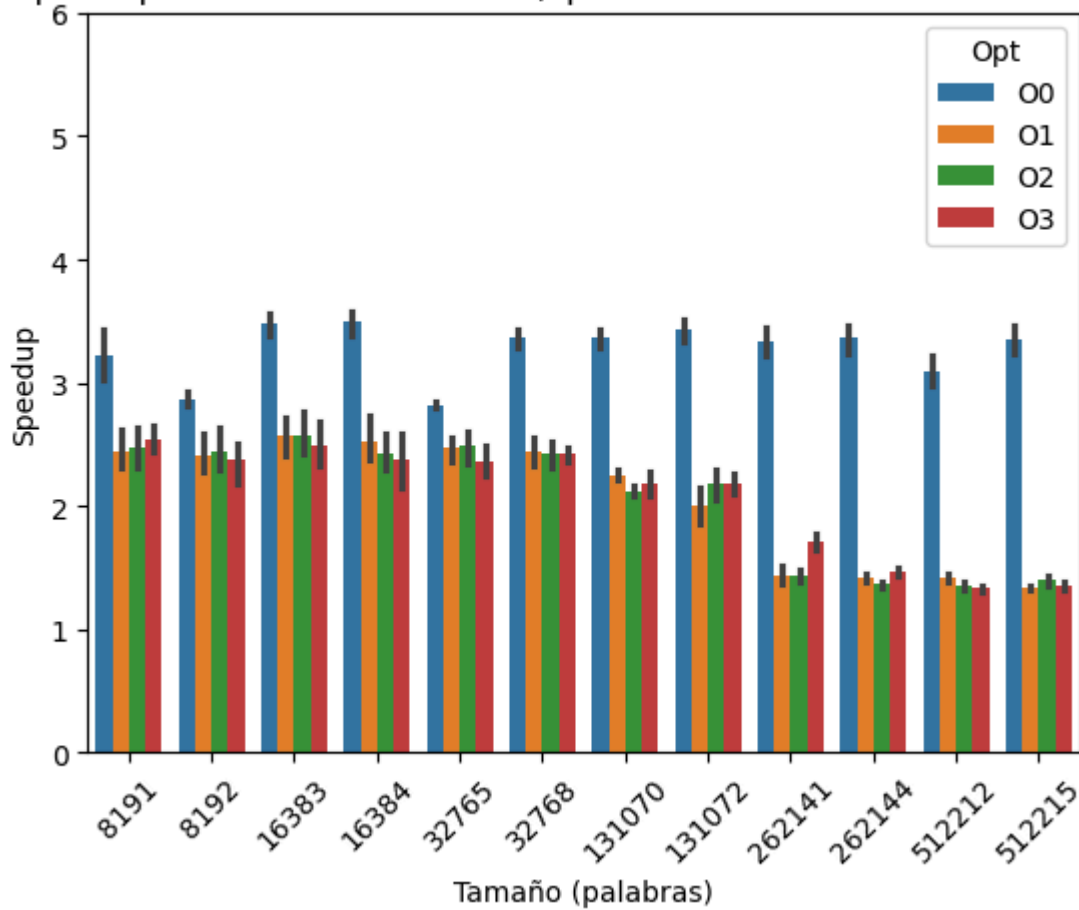
Tiempo (sin SIMD) vs tamaño del vector (optimización O0 vs O1 vs O2 vs O3)



Tiempo AVX vs tamaño del vector (optimización O0 vs O1 vs O2 vs O3)



Speedup vs tamaño del vector (optimización O0 vs O1 vs O2 vs O3)



Observamos unos resultados similares al apartado 2. La principal diferencia es que en este caso el speedup es casi el doble, lo que tiene sentido desde un punto de vista teórico ya que al trabajar con vectores de 8 elementos en vez de 4, podemos hacer uso de un mayor grado de vectorización.

Ejercicio 4

Apartado 0

Análisis de resultados

El programa recibe una o varias imágenes y aplica un filtro de escala de grises. El output, naturalmente, es la imagen proporcionada pero en escala de grises.

También podemos destacar que la resolución de la imagen generada se reduce considerablemente (esto quizá se deba a la precisión del filtro).

Análisis del código

El programa proporcionado procesa secuencialmente las imágenes proporcionadas como input. Inicialmente, se carga la imagen y se guardan parámetros importantes de la misma como el ancho, el alto y el número de canales.

Inmediatamente después reserva memoria para poder crear la nueva imagen con el filtro aplicado. El nombre de la nueva imagen se determina en base a la imagen proporcionada.

Tras crear la nueva imagen, se comprueba que el número de canales sea válido (3 o 4).

Posteriormente, se mide el tiempo de ejecución de la siguiente sección del código, que corresponde a la transformación a escala de grises. Este consiste en un bucle for externo que itera sobre el ancho y un bucle for interno que itera sobre el alto. Para cada posición sobre la que se itera en el bucle interno, se obtiene el píxel correspondiente a la posición determinada por los iteradores, y se aplica un filtro a escala de grises.

A continuación se guarda la imagen generada y se liberan los recursos de la imagen cargada inicialmente. Con esto, se termina la medición del tiempo de ejecución de esta sección del código y se muestra por consola el tiempo transcurrido.

Finalmente, se liberan los recursos de la imagen generada y se procesa la siguiente imagen de la misma manera.

Apartado 1

El bucle denominado en el enunciado como *loop 1* es óptimo para vectorización, aunque requiere de modificaciones tanto para que el compilador lo vectorice automáticamente como para vectorizarlo manualmente con intrínsecos.

La primera modificación necesaria es empezar iterando respecto a la altura y luego respecto a la anchura.

Esto es porque en C, y en muchos otros lenguajes de programación, las matrices se almacenan en memoria en un orden conocido como "row-major order". Esto significa que los elementos de una fila están almacenados en posiciones contiguas en memoria. Por lo tanto, es más eficiente

recorrer primero la altura (filas) y luego la anchura (columnas) porque se accede a los elementos de la matriz de manera secuencial en memoria, lo que mejora la localización de la caché, reduce los fallos de caché y los accesos a memoria.

Apartado 2

Modificamos el bucle que itera sobre cada píxel para que sea acorde a lo expuesto en el apartado 1:

```
for (int j = 0; j < height; j++)
{
    for (int i = 0; i < width; i++)
    {
        getRGB(rgb_image, width, height, 4, i, j, &r, &g, &b);
        grey_image[j * width + i] = (int)(0.2989 * r + 0.5870 *
g + 0.1140 * b);
    }
}
```

Los cambios que hemos realizado respecto al programa original están relacionados con los comentado en apartado 1, y es que hemos aprovechado el modo de almacenar matrices de datos para recorrerlas de modo que los datos se encontrarán alineados aumentando la eficiencia del uso de la caché. Esto lo hemos conseguido simplemente recorriendo las filas de la matriz en vez de las columnas.

Efectivamente al compilar el programa con la opción `-fopt-info-vec-optimized` vemos que en el informe generado con las optimizaciones del compilador se encuentran las siguientes que verifican que el compilador ha sido capaz de vectorizar exitosamente el bucle:

```
greyScale.c:81:31: optimized: loop vectorized using 32 byte vectors
greyScale.c:81:31: optimized: loop versioned for vectorization
because of possible aliasing
greyScale.c:81:31: optimized: loop vectorized using 16 byte vectors
```

Apartado 3

Resolución	Tiempo original (ms)	Tiempo autovectorizado (ms)	Tiempo intrinsics (ms)	FPS original	FPS autovectorizado	FPS intrinsics	Aceleración autovectorizado	Aceleración intrinsics
SD	2,779	1,9614	1,9428	359,84	509,84	514,72	1,42	1,43
HD	9,3974	8,192	7,4416	106,41	122,07	134,38	1,15	1,26
FHD	19,6168	16,2458	15,374	50,98	61,55	65,04	1,21	1,28
4k	88,6184	56,7444	54,029	11,28	17,62	18,51	1,56	1,64
8k	852,634	227,4534	208,7356	1,17	4,40	4,79	3,75	4,08

En la tabla se puede observar cómo a medida que aumenta la resolución el tiempo de procesamiento aumenta para todos los programas y el número de FPS cae drásticamente (son inversamente proporcionales).

El programa original es considerablemente más lento que los otros dos y el programa vectorizado con intrínsecos es ligeramente más rápido que el autovectorizado concordando con lo que debería ocurrir teóricamente.

Por último la aceleración es máxima con la máxima resolución lo que puede sugerir que se saca más partido a la vectorización en tareas más demandantes.

Explicación del algoritmo

El algoritmo original recorre cada píxel mediante un bucle anidado, le asigna un valor en escala de grises en base a una ponderación, y lo almacena como una nueva imagen.

El algoritmo mejorado emplea vectorización con intrínsecos de AVX para procesar varios píxeles simultáneamente, reduciendo el tiempo que necesita para ejecutarse significativamente.

A continuación se desglosan las instrucciones:

1. **Inicialización de los coeficientes.** Los coeficientes para la conversión a escala de grises se inicializan utilizando la instrucción AVX `_mm256_set_ps`, para poder realizar la conversión para varios píxeles a la vez.
2. **Cargar los píxeles.** Dentro del bucle doble, para cada fila (`j`) se toman 4 píxeles (`i += 4`) y se cargan desde la imagen `rgb_image` en dos registros de 128 bits usando `_mm_loadu_si128`. Para esto se emplean dos variables, `lowData` y `highData`, cada una de las cuales maneja 8 bytes.
3. **Conversión a enteros de 32 bits:** Los datos, que son de 8 bits por canal de color, se convierten a enteros de 32 bits sin signo usando `_mm256_cvtepu8_epi32`. Esto permite que cada componente R, G y B pueda utilizarse para operaciones de coma flotante.
4. **Conversión a float.** Se emplea `_mm256_cvtepi32_ps` para convertir los enteros de 32 bits en float. Esto es necesario para realizar la multiplicación para la conversión a escala de grises vectorizada.
5. **Conversión a escala de grises.** Cada canal RGB se multiplica por la ponderación que le corresponde para lograr la conversión a escala de grises. Naturalmente, esto se hace de forma vectorizada, y se emplea la instrucción `_mm256_mul_ps`.
6. **Suma horizontal.** Para obtener el valor final en escala de grises de cada píxel, se realiza una suma horizontal de los canales resultantes de la operación previa usando `_mm256_hadd_ps`, que suma los valores adyacentes, por supuesto de forma vectorial. Esta operación se realiza dos veces para condensar los resultados.
7. **Reordenación.** Tras la suma, se modifican las posiciones de los componentes del vector para que sigan un orden lógico.
8. **Reconversión y guardado.** Los float vuelven a convertirse en enteros de 32 bits, y estos se reducen posteriormente a 8 bits, que es lo que ocupa un canal.